ELEC-H-310 Digital Electronics — Master Synthesis (Logic Circuits, ThS5 → Tr10)

Synthesis of all lectures from ThS5 to Tr10 covering combinational and sequential logic, finite state machines, and hardware description languages (VHDL). The programme is largely cumulative: earlier concepts (boolean algebra, K-maps, combinational circuits) underpin later topics (FSM, pipelines, VHDL). Lecture-specific details are kept in full; foundations are assumed to be known. Notation follows the course slides throughout.

Part I — Number representations and Boolean algebra (ThS5)

1. Numbers representation

Two (signed) numbers — fixed point

Signed representations: sign-magnitude, 1's complement and 2's complement. Interpretation of $b_{n-1}b_{n-2}\ldots b_1b_0$:

- Sign-magnitude: $(-1)^{b_{n-1}} \cdot \sum_{i=0}^{n-2} b_i 2^i$
- 1's complement: $-b_{n-1}(2^{n-1}-1) + \sum_{i=0}^{n-2} b_i 2^i$
- 2's complement: $-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$

General form for any base:

$$\sum_{i=-m}^{n-1} b_i \cdot r^i$$

n integer digits, m fractional digits, base r.

Example: $(1011.101)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 11.625$

Floating point

$$(-1)^s \cdot (1.f) \cdot 2^{e-b}$$

IEEE 754 (simplified): 1 bit sign, 8 bits exponent, 23 bits fraction.

$$(-1)^{b_{31}} \cdot (1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}) \cdot 2^{(\sum_{i=0}^{7} b_{i+23} 2^i) - 127}$$

Arithmetic operations

- Addition: $A + B = A + \overline{B} + 1$
- Subtract: $A - B = A + \overline{B} + 1$
- Mult: $A \cdot B = A \cdot B$
- Div: $A / B = A \cdot \overline{B}$

Overflow: if the sum of two positive or two negative numbers gives a result of the opposite sign.

Extension

Number	Extension
Positive	Add zeros
Negative	Add ones

Truncation

Number	Truncation
Positive	Remove digits
Negative	Remove digits

2. Base conversions

Method of successive divisions — Dec. → Other (integer)

For 53.625, repeatedly divide by the target base until the quotient is 0. $(53)_{10} = (110101)_2$

Fractional part — Dec. → Other

Example: $(0.625)_{10} \rightarrow (0.101)_2$

Operation	Result	Remainder
$0.625 \cdot 2$	1	0.25
$0.25 \cdot 2$	0	0.5
$0.5 \cdot 2$	1	0

Dec. → Octal and Dec. → Hex

Dec. → Octal: $(53.625)_{10} = (65.5)_8$ Dec. → Hex: $(53.625)_{10} = (35.A)_{16}$

Octal → Hex

$(65.5)_8 \rightarrow (110101.101)_2 \rightarrow (35.A)_{16}$

Hex → Octal

$(35.A)_{16} \rightarrow (110101.101)_2 \rightarrow (65.5)_8$

Dec. → Octal and Hex → Dec.

$(65.5)_8 = 6 \cdot 8^1 + 5 \cdot 8^0 + 5 \cdot 8^{-1} = (53.625)_{10}$ $(35.A)_{16} = 3 \cdot 16^1 + 5 \cdot 16^0 + A \cdot 16^{-1} = (53.625)_{10}$

Powers of 2 (binary)

For bases 2, 8, 16, groups blocks that are powers of the base (groups of 1 bit/digit, groups two bits, groups four bits).

Example: $(1011.101)_2 \rightarrow (1.375)_{16}$ $(1011.101)_2 \rightarrow (13.5)_8$

Other bases

Convert from base r_1 to base r_2: $(N)_{r_1} \rightarrow (N)_{10} \rightarrow (N)_{r_2}$

Addition/ Subtraction

Add digit by digit, carry to the next digit. Subtract digit by digit, borrow from the next digit.

3. Boolean algebra

Definition

- $B = \{0, 1\}$
- $x = 0$ if $x = 1$ and vice-versa
- $x \cdot y = 1$ if $x = 1$ and $y = 1$
- $x + y = 1$ if $x = 1$ or $y = 1$

Truth tables for AND, OR, NOT

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

x	$\overline{x}$
0	1
1	0

Xor, xnor, nand, nor

x	y	$x \oplus y$	$\overline{x \oplus y}$	$\overline{x \cdot y}$	$\overline{x + y}$
0	0	0	1	1	1
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	1	0	0

Truth table for MUX, DEMUX

S	Y_0	Y_1
0	X_0	0
1	0	X_1

Truth table for XOR, OR, NOT

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

x	$\overline{x}$
0	1
1	0

Boolean algebra

Commutative	$x \cdot y = y \cdot x$
	$x + y = y + x$
Associative	$(x \cdot y) \cdot z = x \cdot (y \cdot z)$
	$(x + y) + z = x + (y + z)$
Distributive	$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
	$x + (y \cdot z) = (x + y) \cdot (x + z)$
Identity	$x \cdot 1 = x$
	$x + 0 = x$
Null	$x \cdot 0 = 0$
	$x + 1 = 1$
Idempotent	$x \cdot x = x$
	$x + x = x$
Inverse	$x \cdot \overline{x} = 0$
	$x + \overline{x} = 1$
Absorption	$x \cdot (x + y) = x$
	$x + (x \cdot y) = x$
Absorption	$x \cdot (\overline{x} + y) = x \cdot y$
	$x + (\overline{x} \cdot y) = x + y$
Consensus	$(x \cdot y) + (y \cdot z) + (\overline{x} \cdot z) = x \cdot y + \overline{x} \cdot z$
	$(x + y) \cdot (y + z) \cdot (\overline{x} + z) = (x + y) \cdot (\overline{x} + z)$
De Morgan	$\overline{x \cdot y} = \overline{x} + \overline{y}$
	$\overline{x + y} = \overline{x} \cdot \overline{y}$
Involution	$\overline{\overline{x}} = x$

Proof with truth table

Write all the lines of the truth table and compare the columns.

Proof with rewriting

$x + (\overline{x} \cdot y) = (x + \overline{x}) \cdot (x + y) = 1 \cdot (x + y) = x + y$

Shannon decomposition

$$f(x_1, x_2, \ldots, x_n) = x_1 \cdot f(1, x_2, \ldots, x_n) + \overline{x_1} \cdot f(0, x_2, \ldots, x_n)$$

$$f(x_1, x_2, \ldots, x_n) = (x_1 + f(0, x_2, \ldots, x_n)) \cdot (\overline{x_1} + f(1, x_2, \ldots, x_n))$$

Sum of products (SoP)

$$f(x_1, x_2, \ldots, x_n) = \sum_{(a_1, a_2, \ldots, a_n) \in B^n} f(a_1, a_2, \ldots, a_n) \cdot x_1^{a_1} \cdot x_2^{a_2} \cdot \ldots \cdot x_n^{a_n}$$

$$x_i^{a_i} = x_i \text{ if } a_i = 1, \overline{x_i} \text{ if } a_i = 0$$

Product of sums (PoS)

$$f(x_1, x_2, \ldots, x_n) = \prod_{(a_1, a_2, \ldots, a_n) \in B^n} f(a_1, a_2, \ldots, a_n) + x_1^{a_1} + x_2^{a_2} + \ldots + x_n^{a_n}$$

$$x_i^{a_i} = x_i \text{ if } a_i = 0, \overline{x_i} \text{ if } a_i = 1$$

Additional Xnore conversions

- $x \oplus y = \overline{x \oplus y}$
- $x \oplus y = (x + y) \cdot (\overline{x \cdot y})$ Proof: $x \oplus y = (x \cdot \overline{y}) + (\overline{x} \cdot y)$
- $x \oplus y = (x + y) \cdot (\overline{x} + \overline{y})$ Proof: $x \oplus y = (x \cdot \overline{y}) + (\overline{x} \cdot y)$

4. Arithmetic operations

Binary addition of two 1-bit words (half-adder)

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$C = x \cdot y$$

$$S = x \oplus y = (x + y) \cdot (\overline{x \cdot y})$$

The sum bit is exclusive-or.

Full adder

x	y	C_{in}	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$C = (x \cdot y) + (x \cdot C_{in}) + (y \cdot C_{in})$$

$$S = x \oplus y \oplus C_{in}$$

The sum is the **parity** function.

N-bit ripple-carry adder

Each bit is the cascading of a full adder.

Ripple-carry adder

$$C_i = (x_i \cdot y_i) + (x_i \cdot C_{i-1}) + (y_i \cdot C_{i-1})$$

$$S_i = x_i \oplus y_i \oplus C_{i-1}$$

$$C_{out} = C_n$$

5. Negative numbers

2's complement (N)

- Advantages: only one representation, one zero, one sign, one addition/subtraction
- Disadvantages: asymmetric, difficult to read, difficult to multiply

1's complement (N)

- Advantages: easy to read, easy to multiply
- Disadvantages: two representations (one, zero), asymmetric, difficult to add/subtract

Sign-magnitude (N)

- Advantages: symmetric, easy to read, easy to multiply
- Disadvantages: two representations (one, zero), difficult to add/subtract

Excess (N)

- Advantages: only one representation, one zero, one sign, one addition/subtraction
- Disadvantages: difficult to read, difficult to multiply

Powers of 2 (binary)

- Advantages: easy to read, easy to multiply
- Disadvantages: two representations (one, zero), two signs, difficult to add/subtract

Comparison

Number	Sign-mag.	1's	2's	Excess
7	0111	0111	0111	1111
6	0110	0110	0110	1110
5	0101	0101	0101	1101
4	0100	0100	0100	1100
3	0011	0011	0011	1011
2	0010	0010	0010	1010
1	0001	0001	0001	1001
0	0000	0000	0000	1000
-0	1000	1111		
-1	1001	1110	1111	0111
-2	1010	1101	1110	0110
-3	1011	1100	1101	0101
-4	1100	1011	1100	0100
-5	1101	1010	1011	0011
-6	1110	1001	1010	0010
-7	1111	1000	1001	0001
-8			1000	0000

2's complement (-8)

Negative: $\overline{x} + 1$ $\overline{(x - 1)}$ $\overline{(x + 1)}$ $\overline{(x \ll 1)}$ $\overline{(x \gg 1)}$ The first and the **last** bits are the same.

1's complement (-7)

Negative: $\overline{x}$ $\overline{(x + 1)}$ $\overline{(x - 1)}$ $\overline{(x \ll 1)}$ $\overline{(x \gg 1)}$ The first and the **last** bits are the same.

Sign-magnitude (-7)

Negative: $\overline{x}$ $\overline{(x + 1)}$ $\overline{(x - 1)}$ $\overline{(x \ll 1)}$ $\overline{(x \gg 1)}$ The first and the **last** bits are the same.

Excess (-8)

Negative: $\overline{x}$ $\overline{(x + 1)}$ $\overline{(x - 1)}$ $\overline{(x \ll 1)}$ $\overline{(x \gg 1)}$ The first and the **last** bits are the same.

Powers of 2 (binary)

Negative: $\overline{x}$ $\overline{(x + 1)}$ $\overline{(x - 1)}$ $\overline{(x \ll 1)}$ $\overline{(x \gg 1)}$ The first and the **last** bits are the same.

6. Combinational circuits

2:1 multiplexer — 2-nor/nand-nor

- Nand-nand: $(x \cdot \overline{s}) + (y \cdot s)$
- Nand-nand: $(x + \overline{s}) \cdot (y + s)$
- Nand-nand: $(x \cdot \overline{s}) + (y \cdot s) + (x \cdot y)$
- Nand-nand: $(x + \overline{s}) \cdot (y + s) \cdot (x + y)$

Decoder

- 2:4 decoder: $Y_0 = \overline{x} \cdot \overline{y}$ $Y_1 = \overline{x} \cdot y$ $Y_2 = x \cdot \overline{y}$ $Y_3 = x \cdot y$
- 3:8 decoder: $Y_0 = \overline{x} \cdot \overline{y} \cdot \overline{z}$ $Y_1 = \overline{x} \cdot \overline{y} \cdot z$ $Y_2 = \overline{x} \cdot y \cdot \overline{z}$ $Y_3 = \overline{x} \cdot y \cdot z$ $Y_4 = x \cdot \overline{y} \cdot \overline{z}$ $Y_5 = x \cdot \overline{y} \cdot z$ $Y_6 = x \cdot y \cdot \overline{z}$ $Y_7 = x \cdot y \cdot z$

Encoding

- 4:2 encoder: $x = Y_2 + Y_3$ $y = Y_1 + Y_3$
- 8:3 encoder: $x = Y_4 + Y_5 + Y_6 + Y_7$ $y = Y_2 + Y_3 + Y_6 + Y_7$ $z = Y_1 + Y_3 + Y_5 + Y_7$

Demultiplexer

- 1:2 demultiplexer: $Y_0 = x \cdot \overline{s}$ $Y_1 = x \cdot s$
- 1:4 demultiplexer: $Y_0 = x \cdot \overline{s_1} \cdot \overline{s_0}$ $Y_1 = x \cdot \overline{s_1} \cdot s_0$ $Y_2 = x \cdot s_1 \cdot \overline{s_0}$ $Y_3 = x \cdot s_1 \cdot s_0$

Multiplexer

- 4:1 multiplexer: $Y = x_0 \cdot \overline{s_1} \cdot \overline{s_0} + x_1 \cdot \overline{s_1} \cdot s_0 + x_2 \cdot s_1 \cdot \overline{s_0} + x_3 \cdot s_1 \cdot s_0$
- 8:1 multiplexer: $Y = x_0 \cdot \overline{s_2} \cdot \overline{s_1} \cdot \overline{s_0} + x_1 \cdot \overline{s_2} \cdot \overline{s_1} \cdot s_0 + x_2 \cdot \overline{s_2} \cdot s_1 \cdot \overline{s_0} + x_3 \cdot \overline{s_2} \cdot s_1 \cdot s_0 + x_4 \cdot s_2 \cdot \overline{s_1} \cdot \overline{s_0} + x_5 \cdot s_2 \cdot \overline{s_1} \cdot s_0 + x_6 \cdot s_2 \cdot s_1 \cdot \overline{s_0} + x_7 \cdot s_2 \cdot s_1 \cdot s_0$

Implementation

- 2:1 multiplexer: $(x \cdot \overline{s}) + (y \cdot s)$
- 2:4 decoder: $Y_0 = \overline{x} \cdot \overline{y}$ $Y_1 = \overline{x} \cdot y$ $Y_2 = x \cdot \overline{y}$ $Y_3 = x \cdot y$
- 4:1 multiplexer: $(x_0 \cdot \overline{s_1} \cdot \overline{s_0}) + (x_1 \cdot \overline{s_1} \cdot s_0) + (x_2 \cdot s_1 \cdot \overline{s_0}) + (x_3 \cdot s_1 \cdot s_0)$

Part II — Combinational logic synthesis, hazards, intro to sequential (ThS7)

1. Representing logic functions — Shannon forms

1. Truth tables (TTs)

- Pro: canonical representation, easy to read
- Con: exponential in inputs, impractical, hard to read for more than 3 inputs

2. Sum of products (SoP) / Product of sums (PoS)

- Pro: canonical representation, easy to read, easy to implement, easy to manipulate
- Con: exponential in inputs, not minimal, not unique

3. Shannon decomposition

- Pro: $f(x_1, x_2, \ldots, x_n) = x_1 \cdot f(1, x_2, \ldots, x_n) + \overline{x_1} \cdot f(0, x_2, \ldots, x_n)$
- Con: $f(x_1, x_2, \ldots, x_n) = (x_1 + f(0, x_2, \ldots, x_n)) \cdot (\overline{x_1} + f(1, x_2, \ldots, x_n))$

4. Binary decision diagrams (BDDs)

- Pro: canonical representation, compact representation, easy to manipulate
- Con: not minimal, not unique

5. Boolean expressions

- Pro: compact representation, not unique
- Con: not canonical, difficult to manipulate

Shannon expansion

$$f(x_1, x_2, \ldots, x_n) = x_1 \cdot f(1, x_2, \ldots, x_n) + \overline{x_1} \cdot f(0, x_2, \ldots, x_n)$$

$$f(x_1, x_2, \ldots, x_n) = (x_1 + f(0, x_2, \ldots, x_n)) \cdot (\overline{x_1} + f(1, x_2, \ldots, x_n))$$

2. Karnaugh maps

Rectangle

- Contains a power-of-two number of cells
- Cells are adjacent
- Opposite cells are adjacent

Implicant

- Rectangle
- Contains only 1s

Prime implicant

- Implicant
- Cannot be extended to a larger implicant

Essential prime implicant

- Prime implicant
- Contains a 1 that is not covered by any other prime implicant

Procedure

- Draw the K-map
- Find essential prime implicants
- Find other prime implicants to cover the remaining 1s

Don't care

- Can be used as either 0 or 1

3. Combinational hazards

Static-1 hazard

- Two adjacent 1s are not covered by the same implicant
- Glitch: 1 → 0 → 1

Static-0 hazard

- Two adjacent 0s are not covered by the same implicant
- Glitch: 0 → 1 → 0

Dynamic hazard

- Three adjacent 1s are not covered by the same implicant
- Glitch: 1 → 0 → 1 → 0 or 0 → 1 → 0 → 1

Fixes

- Add redundant implicants
- Use Shannon decomposition

4. Sequential logic — intro

Latch

- Level-sensitive
- Transparent when enabled
- Output changes immediately when input changes

Flip-flop

- Edge-triggered
- Samples input on clock edge
- Output changes after clock edge

Setup time

- Minimum time before the clock edge that the input must be stable

Hold time

- Minimum time after the clock edge that the input must be stable

Propagation delay

- Time from the clock edge to a stable output

Clock skew

- Difference between clock arrival times at different flip-flops

Metastability

- Flip-flop enters a state between logic levels and may take a long time to resolve

Synchronizer

- Two flip-flops in series to reduce the probability of metastability propagation

Part III — Combinational logic synthesis, hazards, intro to sequential (ThS7)

1. Combinational logic synthesis from verbal specifications

The synthesis problem

- Translate an informal description, verbal specification into a formal one. Because ambiguous, incomplete, contradictory.

Example: 2-bit comparator

- Inputs: two 2-bit numbers A and B
- Outputs: 1 if A > B, 1 if A = B, 1 if A < B

Formalization

- Truth table
- Boolean equations
- Logic gates
- Choose a target technology: gates, LUTs, PLAs, standard cells, custom cells, gate arrays, FPGAs, ASICs, etc.

Two-level logic

- Any boolean function can be expressed in two levels: AND-OR (SoP) or OR-AND (PoS)
- Gates: AND, OR, NOT
- Delay: 3 gate delays
- Pros: simple, fast, predictable
- Cons: exponential in inputs, not practical for large functions

Multi-level logic

- Any boolean function can be expressed in multiple levels of AND, OR, NOT gates
- Pros: practical, scalable, smaller than two-level
- Cons: variable delay, harder to analyze

Cost metrics

- Gate count
- Literal count (SoP literals)
- Delay (critical path)
- Area
- Power

Optimization goals

- Minimize cost
- Minimize delay
- Minimize power
- Meet constraints

Don't care

- Inputs that never occur
- Outputs that are ignored
- Can be used to simplify logic

2. Two-level logic minimization

Quine-McCluskey

- Exact minimization
- Finds all prime implicants
- Finds minimum cover
- Exponential in inputs

Espresso

- Heuristic minimization
- Fast
- Near-optimal
- Handles large functions

Implicant

- Product term
- Covers some minterms

Prime implicant

- Implicant
- Cannot be combined with another implicant to form a larger implicant

Essential prime implicant

- Prime implicant
- Covers a minterm that no other prime implicant covers

Procedure

- Find prime implicants
- Find essential prime implicants
- Cover remaining minterms

3. Multi-level logic

Advantages

- Smaller than two-level
- Faster than two-level
- Easier to implement with standard cells

Disadvantages

- Variable delay
- Harder to analyze
- Harder to test

Factoring

- $ab + ac \rightarrow a(b + c)$

Decomposition

- $f(a, b, c, d) = g(a, b) + h(c, d)$

Extraction

- Find common subexpressions
- Share them

Substitution

- Express one function in terms of another

Simplification

- Apply two-level minimization locally

Technology mapping

- Map to gates
- Map to LUTs
- Map to standard cells

4. Logic optimization using K-Maps

Why optimize

- Fewer gates → smaller, faster, cheaper, less power, more reliable (fewer gates and fewer inputs per gate)
- Two-level logic: AND-OR (SoP) or OR-AND (PoS). Any boolean function can be expressed in two levels.

Prime implicant

- Implicant
- Cannot be combined with another implicant to form a larger implicant

Essential prime implicant

- Prime implicant
- Covers a minterm that no other prime implicant covers

Procedure

- Find prime implicants
- Find essential prime implicants
- Cover remaining minterms

Don't care

- Inputs that never occur
- Outputs that are ignored
- Can be used to simplify logic

5. Incompletely specified logic functions

"Don't care" set — Yves' example

- Start: $\{000, 001, 101, 111\}$
- Expand: $\{0-0, 0-1, -01, 1-1\}$
- Cover: $\{0-0, -01, 1-1\}$

Espresso

- Heuristic minimization
- Fast
- Near-optimal
- Handles large functions

Espresso-exact

- Exact minimization
- Finds all prime implicants
- Finds minimum cover
- Exponential in inputs

Espresso-strong

- Heuristic minimization
- Tries harder than Espresso
- Still not optimal

Espresso-exact — Yves' example

- Prime implicants: $\{0-0, -01, 1-1\}$
- Essential prime implicants: $\{0-0, 1-1\}$
- Cover: $\{0-0, 1-1, -01\}$

Espresso-strong — Yves' example

- Start: $\{000, 001, 101, 111\}$
- Expand: $\{0-0, 0-1, -01, 1-1\}$
- Cover: $\{0-0, -01, 1-1\}$

2. Synthesis of sider circuits

Full table — derived equations

x_1	x_2	x_3	y_1	y_2	y_3
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	1	1	1

Full table — derived equations

x_1	x_2	x_3	y_1	y_2	y_3
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	1	1	1

Kraft-McMillan inequality

$$\sum_{i=1}^{n} 2^{-l_i} \leq 1$$

Equality means the code is complete; strict means it is incomplete (redundant). Fixed-length codes satisfy this with equality.

Using beyond Kraft

Kraft-McMillan tells you if a prefix code exists for a single architecture (one alphabet, one D). Useful when comparing encoders across different alphabets or when checking whether a proposed set of codewords is valid.

3. Circuits with multiple outputs

Example 1 — 7-segment display

Ten digits (0–9), seven outputs (a, b, c, d, e, f, g), inputs x_1, x_2, x_3, x_4 with x_1 = MSB.

Decimal	BCD (8421)	Excess-3 (XS3)
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Excess-3 is a self-complementary code. Optimal functions with don't cares for 10–15:

$$a = x_1 + x_3 + x_2x_4 + x_2x_1 + x_4x_1$$
$$b = x_2 + x_3x_4 + x_3x_1$$
$$c = x_2 + x_3 + x_4 + x_1$$
$$d = x_3x_4 + x_2x_3x_1 + x_2x_4x_1 + x_3x_1$$
$$e = x_2x_4 + x_3x_4 + x_1$$
$$f = x_1 + x_2x_3 + x_2x_4 + x_3x_4$$
$$g = x_1 + x_2x_3 + x_2x_4 + x_3x_4$$

Example 2 — ALU — Arithmetic and logic unit

Inputs: x_1, x_2 — operation code; x_3, x_4 — operands.

x_1	x_2	Operation
0	0	Addition
0	1	Subtraction
1	0	Bitwise AND
1	1	Bitwise XOR

$$z_1 = x_3 + x_4$$
$$z_2 = x_3 + x_4 + x_1$$
$$z_3 = x_3x_4 + x_3x_1 + x_4x_1$$
$$z_4 = x_3x_4 + x_3x_1 + x_4x_1 + x_2$$

4. Hazards

The traffic of ideas

Boolean algebra assumes instant relay. Implementations disagree.

Three wires carrying 0 and 1 arrive at a gate. The output should be steady. If the inputs cross at different times, the output may flicker. That flicker is a hazard.

Glitches. Causes. Fixes.

A hazard is a glitch caused by a race. Two paths to one gate arrive at different times. Hazards are transient. They do not change the logic. They can trigger downstream circuits. Fixes: add redundant terms, use synchronous clocking, or design asynchronously.

Part IV — Synthesis of sequential circuits (TK66)

1. Sequential systems — classes & representation

Why sequential

Combinational circuits forget everything. Sequential circuits remember. Memory enables sequences, protocols, and computation.

Memory: present or future

This chapter covers two kinds of memory. Memory that holds signal y^T at the present T. Memory that stores a future signal y^{T+1} for one step. Synthesis is combinatorial. Timing is sequential.

The classes of sequential systems

- **Moore machine** — output is a combinational function of state variables only.
- **Mealy machine** — output is a combinational function of state variables and inputs.

Representations

- **Equations** — Moore, Mealy, and D-synthesis.
- **Table** — state transition table.
- **Diagram** — state transition diagram.

Notation

- x_i — input variables.
- y_i — state variables.
- Y_i — next state functions.
- z_i — output functions.

Encoding (1 - 16, 17 - 32, 33 - 48, 49 - 64) — state assignment

Logic synthesis

Final circuit — the output of the synthesis process, including $\overline{\text{CLK}}$, $\overline{\text{PRE}}$, and $\overline{\text{CLR}}$ (or $\overline{\text{CS}}$ and $\overline{\text{RESET}}$) pins, ready for implementation.

2. Moore machines

Example: Mod-3 counter

Two state variables y_1, y_2. Three in use, one unused. Moore output: $z = y_1 + y_2$.

y_1	y_2	Y_1	Y_2	z
0	0	0	1	0
0	1	1	0	1
1	0	0	0	1
1	1	—	—	—

$$Y_1 = y_1\overline{y_2}$$
$$Y_2 = \overline{y_1}\,\overline{y_2}$$

State table for Moore machines

Moore output depends on state only. Table 2 adds a column. Table 1 cannot. Synthesis uses Table 1 and the output equation. Table 2 is for simulation and verification.

y_1	y_2	Y_1	Y_2	z
0	0	0	1	0
0	1	1	0	1
1	0	0	0	1
1	1	—	—	—

Encoding (1 - 16, 17 - 32, 33 - 48, 49 - 64) — state assignment

Logic synthesis

Final circuit — the output of the synthesis process, including $\overline{\text{CLK}}$, $\overline{\text{PRE}}$, and $\overline{\text{CLR}}$ (or $\overline{\text{CS}}$ and $\overline{\text{RESET}}$) pins, ready for implementation.

3. Mealy machines

Example: Mod-3 counter

Two state variables y_1, y_2. Three in use, one unused. Mealy output: $z = y_1 + y_2 + x$.

x	y_1	y_2	Y_1	Y_2	z
0	0	0	0	1	0
0	0	1	1	0	1
0	1	0	0	0	1
0	1	1	—	—	—
1	0	0	0	0	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	1	—	—	—

$$Y_1 = y_1\overline{y_2}\,\overline{x} + \overline{y_1}\,y_2\,x$$
$$Y_2 = \overline{y_1}\,\overline{y_2}\,\overline{x} + y_1\overline{y_2}\,x$$

Mealy state table

Output depends on state and input. Table 1 is compact. Table 2 shows the full picture.

x	y_1	y_2	Y_1	Y_2	z
0	0	0	0	1	0
0	0	1	1	0	1
0	1	0	0	0	1
0	1	1	—	—	—
1	0	0	0	0	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	1	—	—	—

Encoding (1 - 16, 17 - 32, 33 - 48, 49 - 64) — state assignment

Logic synthesis

Final circuit — the output of the synthesis process, including $\overline{\text{CLK}}$, $\overline{\text{PRE}}$, and $\overline{\text{CLR}}$ (or $\overline{\text{CS}}$ and $\overline{\text{RESET}}$) pins, ready for implementation.

4. Optimization of primitive state tables

Redundant states — intro

- **Equivalent states** — indistinguishable from outside. Merge them.
- **Redundant states** — more than needed. Remove them.

Fewer states, simpler circuit. Fewer flip-flops, fewer gates, fewer transitions.

Example: Mealy machine

Two inputs, one output. 9 states.

Present state	$x = 0$		$x = 1$	
	Next state	z	Next state	z
a	a	0	b	0
b	c	0	d	0
c	e	0	f	0
d	g	0	h	0
e	a	1	b	1
f	c	1	d	1
g	e	1	f	1
h	g	1	h	1

Partition refinement

Start with two groups: outputs 0 and outputs 1. Split groups when states disagree on where inputs lead. Stop when nothing splits. Merge states in the same group.

Implication table

Alternative method. Pairs of states. Mark incompatible pairs. Propagate implications. Merge unmarked pairs.

Incompletely specified machines

Some entries are don't care. Choose values to maximize merging. Then minimize.

5. Synthesis of Moore & Mealy machines — full example

Specification

Two inputs, one output. Detect 0110 or 1001. Overlapping allowed. Moore or Mealy.

Moore state diagram

Moore state table

Present state	$x = 0$		$x = 1$	
	Next state	z	Next state	z
a	a	0	b	0
b	c	0	d	0
c	e	0	f	0
d	g	0	h	0
e	a	1	b	1
f	c	1	d	1
g	e	1	f	1
h	g	1	h	1

Mealy state table

Output depends on state and input. Table 1 is compact. Table 2 shows the full picture.

x	y_1	y_2	Y_1	Y_2	z
0	0	0	0	1	0
0	0	1	1	0	1
0	1	0	0	0	1
0	1	1	—	—	—
1	0	0	0	0	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	1	—	—	—

Encoding (1 - 16, 17 - 32, 33 - 48, 49 - 64) — state assignment

Logic synthesis

Final circuit — the output of the synthesis process, including $\overline{\text{CLK}}$, $\overline{\text{PRE}}$, and $\overline{\text{CLR}}$ (or $\overline{\text{CS}}$ and $\overline{\text{RESET}}$) pins, ready for implementation.

Part V — Asynchronous sequential circuits (TK66)

1. Race conditions

Previous synthesis

Clock ticks. Flip-flops change. Wait for the dust to settle. Then sample the next state. Synchronization hides the chaos. Now we remove the clock. Feedback wires carry signals directly. Gates race. Delays decide who wins.

Feedback

Signals from later stages return to earlier stages. Feedback creates memory. Feedback creates instability. Feedback creates oscillation.

Race conditions — example

Two inputs, x_1 and x_2. Two state variables, y_1 and y_2. Two outputs, z_1 and z_2.

Stable states

- **Equilibrium** — the system settles. Inputs and state stop changing. Outputs stop changing.
- **Stable state** — $Y_i = y_i$. The next state equals the current state.

Unstable states

- **Transition** — $Y_i \neq y_i$. The next state differs from the current state.
- **Oscillation** — $Y_i = \overline{y_i}$. The next state is the opposite. Flip-flop mode.

Transition table

Inputs	$y_1y_2 = 00$	$y_1y_2 = 01$	$y_1y_2 = 11$	$y_1y_2 = 10$
$x_1x_2 = 00$	00	01	11	10
$x_1x_2 = 01$	01	01	11	11
$x_1x_2 = 11$	11	01	11	10
$x_1x_2 = 10$	10	00	11	10

Critical race

Two state variables change. The order matters. The final state depends on who wins. Different orders lead to different stable states. Unpredictable.

Non-critical race

Two state variables change. The order does not matter. All orders lead to the same stable state. Predictable.

Essential hazard

Input changes. Feedback delays differ. Wrong path wins. Temporary wrong state. Can trigger downstream errors.

Fixes

- Slow the feedback.
- Speed up the input.
- Add redundant terms.
- Use synchronous clocking.

2. Synchronous & asynchronous circuits — recap and review

Recap

Clocks simplify everything. Synchronization hides the chaos. Feedback makes it visible.

Asynchronous circuits — performance

Clocks waste time. Every operation waits for the slowest. Asynchronous circuits can be faster. Only the actual delay matters. Average-case performance beats worst-case.

Asynchronous circuits — advantages

- Lower peak power. Fewer transitions. No clock distribution network.
- Modular composition. Handshake protocols. Easier than global timing analysis.
- Graceful degradation. Slow down instead of failing.

Asynchronous circuits — challenges

- Race conditions. Hazards. Glitches.
- Testing. Fewer tools. Less mature ecosystem.
- Debugging. Harder to reproduce. Timing-dependent bugs.

Synchronous design

Clock arrives. Flip-flops change. Wait for the dust to settle. Sample the next state. Synchronization hides the chaos.

Asynchronous design

Remove the clock. Feedback wires carry signals directly. Gates race. Delays decide who wins. Feedback makes instability visible.

Synchronization

Synchronizers convert asynchronous inputs into synchronous outputs. Metastability is the failure mode. Synchronizers reduce the probability of failure.

3. Synchronizers

Synchronizer basics

Asynchronous input. Synchronizer. Synchronous output.

- **Metastable** — neither 0 nor 1. Voltage between thresholds.
- **Resolution** — drifting toward 0 or 1. Takes time.
- **Failure** — downstream logic sees the wrong value. Propagation of metastability.

Mean time between failures (MTBF)

$$\text{MTBF} = \frac{e^{t_r/\tau}}{f_c \cdot f_d \cdot T_w}$$

- t_r — resolution time.
- τ — metastable time constant.
- f_c — clock frequency.
- f_d — data frequency.
- T_w — window time.

Exponential in resolution time. Linear in everything else. Doubling resolution time squares the failure rate.

Two-flip-flop synchronizer

Two flip-flops in series. Two clock periods of resolution. Failure rate drops exponentially. Standard practice for most designs.

Three-flip-flop synchronizer

Three flip-flops in series. Three clock periods of resolution. Failure rate drops further. Used when two flip-flops are not enough.

Synchronizer with feedback

Synchronizer output feeds back to the input. Forces the input to a stable value. Reduces the probability of metastable input.

Synchronizer with handshake

Handshake protocol. Request. Acknowledge. Synchronizer inside the protocol. Asynchronous interface. Synchronous internals.

Synchronizer with FIFO

First-in, first-out buffer. Decouples producer and consumer. Absorbs rate mismatch. Synchronizer at each boundary.

4. Two-flip-flop synchronizer

Specification

Two inputs, x_1 and x_2. Two outputs, z_1 and z_2.

- z_1 — synchronized input. Two clock periods behind x_1.
- z_2 — edge detector. Pulses when x_1 changes.

Timing diagram

Truth table

Inputs	$y_1y_2 = 00$	$y_1y_2 = 01$	$y_1y_2 = 11$	$y_1y_2 = 10$
$x_1x_2 = 00$	00	01	11	10
$x_1x_2 = 01$	01	01	11	11
$x_1x_2 = 11$	11	01	11	10
$x_1x_2 = 10$	10	00	11	10

Implementation

5. Pulse synchronizer

Pulse synchronizer

Input pulse. Output pulse. Two clock periods. Edge detector. Two-flip-flop synchronizer. Pulse generator.

Pulse synchronizer — example

6. Synchronizer failure

Failure

Metastable input. Resolution time exceeds clock period. Downstream logic sees the wrong value. Propagation of metastability. Rare but not zero.

Mean time between failures (MTBF)

$$\text{MTBF} = \frac{e^{t_r/\tau}}{f_c \cdot f_d \cdot T_w}$$

- t_r — resolution time.
- τ — metastable time constant.
- f_c — clock frequency.
- f_d — data frequency.
- T_w — window time.

Exponential in resolution time. Linear in everything else. Doubling resolution time squares the failure rate.

Synchronizer with feedback

Synchronizer output feeds back to the input. Forces the input to a stable value. Reduces the probability of metastable input.

Synchronizer with handshake

Handshake protocol. Request. Acknowledge. Synchronizer inside the protocol. Asynchronous interface. Synchronous internals.

Synchronizer with FIFO

First-in, first-out buffer. Decouples producer and consumer. Absorbs rate mismatch. Synchronizer at each boundary.

7. Synchronizer design

Design

Two-flip-flop synchronizer. Three-flip-flop synchronizer. Pulse synchronizer. Feedback synchronizer. Handshake synchronizer. FIFO synchronizer.

Design rules

- Two flip-flops minimum.
- Three flip-flops for critical paths.
- Pulse synchronizer for edge detection.
- Feedback synchronizer for slow inputs.
- Handshake synchronizer for multi-bit data.
- FIFO synchronizer for rate mismatch.
- Synchronizer at every boundary.

Testing

Simulation. Formal verification. Stress testing. Fault injection. Long-run statistics.

Pitfalls

Too few flip-flops. Too many flip-flops. Combinational logic before the synchronizer. Wrong clock. Missing synchronizer. Wrong synchronizer. Untested failure mode.

Checklist

Two flip-flops minimum. Three flip-flops for critical paths. Pulse synchronizer for edge detection. Feedback synchronizer for slow inputs. Handshake synchronizer for multi-bit data. FIFO synchronizer for rate mismatch. Synchronizer at every boundary. Combinational logic before the synchronizer. Wrong clock. Missing synchronizer. Wrong synchronizer. Untested failure mode.

Total points (77 mandatory plus 10 bonus): 87 (0 , 0 , 0 ,